Fieldwork

Freelancer Client Portal — Project Build Summary

A portfolio SaaS project built end-to-end in a single session,

from an empty repository to a running, verified application.

Stack: Next.js 16 · TypeScript · Prisma · PostgreSQL · NextAuth v5 · Tailwind + shadcn/ui

1. Overview

This document summarizes the build of Fieldwork, a portfolio SaaS application for freelancers to manage clients, track projects, generate invoices, share files, and run a subscription-gated client portal.

The requested features were:

- User authentication
- Dashboard
- Client management
- Project tracking
- Invoice generation (PDF)
- File uploads
- Stripe subscriptions (Free/Pro)
- Role-based permissions

All eight were implemented, verified working end-to-end, and are running locally.

2. Planning & Decisions

A few foundational questions were resolved before writing any code, since they determine the shape of everything downstream:

- **Stack:** Next.js (App Router) + TypeScript — a single full-stack codebase rather than a separate frontend/back-end.
- **Database:** PostgreSQL via Prisma, run locally through Docker Compose.
- **File storage:** local disk for development, with a documented path to swap in S3-compatible storage for production.
-

Stripe: the user confirmed this was for portfolio purposes only, so billing was simulated (a Free/Pro toggle with an explicit “this is a demo” confirmation dialog) rather than integrating real Stripe Checkout and webhooks.

A formal implementation plan was written and approved before any code was written, covering the data model, route structure, and a phased build order.

3. Architecture

Tech stack

- Next.js 16 (App Router, Turbopack)
- TypeScript
- Prisma ORM + PostgreSQL (Docker Compose locally)
- NextAuth.js (Auth.js) v5 — credentials provider, bcrypt hashing, JWT sessions
- Tailwind CSS v4 + shadcn/ui (Base UI primitives under the hood)
- React Hook Form + Zod for form state and validation
- @react-pdf/renderer for server-side invoice PDF generation
- Recharts for the dashboard revenue chart

Data model (Prisma)

- User — account holders (team members and client-portal users share this table)
- Account — a freelancer's workspace; holds the plan (FREE / PRO)
- Membership — links a User to an Account with a role: OWNER, ADMIN, or MEMBER
- Client — a freelancer's client, scoped to an Account
- ClientPortalAccess — links a User to exactly one Client, granting scoped read-only access
- Project — belongs to an Account and a Client
- Invoice / InvoiceItem — line-item invoices with computed subtotal / tax / total
- FileUpload — files attached to a client and/or project
- Invite — pending invitations, used for both team invites and client-portal invites

Route structure

- / — marketing landing page
- /login, /register, /invite/[token] — auth flows
- /dashboard/* — the team-facing app (clients, projects, invoices, team, billing)
- /portal/* — the read-only client-facing app
- /api/uploads, /api/uploads/[id] — file upload and secure download

Route protection is centralized in proxy.ts (Next.js 16's renamed middleware.ts), which redirects unauthenticated users to /login and keeps team users and client-portal users confined to their respective route trees.

Permission model

Role checks are centralized in lib/permissions.ts, which every dashboard/portal page and server action calls before touching data.

Role	Can do
OWNER	Everything, including billing and removing admins
ADMIN	Manage clients/projects/invoices/files, invite team members and clients
MEMBER	Manage clients/projects/invoices/files, no team/billing access
CLIENT (portal)	Read-only view of their own projects/invoices, scoped to one client

4. What Was Built

- **Authentication —** registration (creates a User + a new Account + OWNER membership in one transaction), credentials login, and an invite-acceptance flow handling both brand-new and existing users.
- **Dashboard —** client/active-project counts, outstanding invoice total, this-month revenue, a 6-month revenue chart, and a recent-invoices table, all from real Prisma queries.
- **Client management —** full CRUD with a create/edit dialog, a detail page with tabs for projects/invoices/files, and a portal-access panel.
- **Project tracking —** full CRUD, status filtering, budget and date tracking, linked back to its client.
- **File uploads —** uploaded to local disk outside /public, with size/extension validation and permission-gated downloads.
- **Invoicing —** a line-item builder with live-calculated totals, auto-generated invoice numbers, a status workflow, and on-demand PDF generation available to both the team and the client portal.
- **Team management —** invite teammates by email/role, promote/demote Admin”Member, remove members.
- **Client portal invites —** a Pro-only feature granting a client read-only access to exactly one client's data.
- **Mock billing —** Free/Pro plan cards with a demo-labeled confirmation dialog and real plan-gated limits (Free capped at 3 clients / 5 projects).
- **Landing page —** a marketing page describing the product and its two pricing tiers.

5. Problems Encountered & How They Were Fixed

This section is the most useful part for understanding what actually happened during the build — most of these were not obvious until they surfaced.

> ### 5.1 The project folder name broke create-next-app
>
> **Problem:** The working directory "SaaS-site" contains capital letters, which npm’s package-name rules reject, so create-next-app refused to scaffold directly into it.
>
> **Fix:** Scaffolded into a temporary correctly-named subfolder ("freelancer-portal"), then moved all generated files (including the hidden .git) up into the project root and removed the now-empty subfolder.

5.2 Prisma 7 broke the schema’s datasource url

Problem: npm install prisma pulled the latest major version (7.x), which removed support for "url = env(“DATABASE_URL”)" directly in schema.prisma in favor of a new prisma.config.ts + driver-adapter setup — a breaking change that surfaced immediately as a schema validation error on prisma generate.

Fix: Pinned prisma and @prisma/client to the last stable 6.x release (6.19.3), which keeps the simpler, well-documented schema-based configuration appropriate for this project's scope.

5.3 shadcn/ui now generates Base UI components, not Radix — asChild doesn’t exist

Problem: The installed shadcn CLI generated components built on @base-ui/react instead of the more familiar Radix UI primitives. Base UI uses a different polymorphism API (render={<Element />}) instead of the asChild + wrapped-child pattern used throughout the initial build (buttons-as-links, dialog/alert-dialog/sheet triggers). This produced a wall of TypeScript errors once the whole app was type-checked.

Fix: Inspected shadcn's own generated source (dialog.tsx, alert-dialog.tsx) to confirm the correct pattern, then systematically converted every asChild / wrapped-child usage across 14 files to the render={<Component {...props} />} form, with children passed directly to the trigger/parent component instead of nested inside the rendered element.

5.4 NextAuth v5 type augmentation silently didn’t apply

Problem: Custom session/JWT fields (accountId, role, clientId) were declared via the standard declare module "next-auth" pattern, but TypeScript kept reporting "Type ‘unknown’ is not assignable..." on those fields — the augmentation wasn’t merging.

Fix: Traced it to the fact that next-auth v5 doesn't declare Session/JWT itself — it re-exports them from @auth/core/types and @auth/core/jwt. TypeScript's declaration merging only works against the module where an interface is originally declared, so the augmentation had to target @auth/core/types and @auth/core/jwt directly instead of next-auth / next-auth/jwt.

5.5 A TypeScript overload-resolution trap in an API route

Problem: A helper function typed a session parameter as Awaited<ReturnType<typeof auth>>. Because auth from NextAuth v5 is an overloaded function (usable as auth() for a session, or auth(handler) as middleware), TypeScript resolved ReturnType against the last overload signature (the middleware one) instead of the plain session-fetching one, producing nonsensical errors like “Property 'user' does not exist on type 'NextMiddleware'.”

Fix: Replaced the inferred type with an explicit Session | null (imported from next-auth), sidestepping the overload ambiguity entirely.

5.6 Zod v4 + React Hook Form resolver type mismatch

Problem: The invoice form's Zod schema used z.coerce.number() for quantity/price fields (so number <input>s, which hand back strings, still validate correctly). This created a legitimate difference between the schema's 'input' type (pre-coercion) and 'output' type (post-coercion) that @hookform/resolvers' zodResolver typing didn't reconcile automatically, breaking useForm's generic inference.

Fix: Cast the resolver to the form's output type (zodResolver(invoiceSchema) as Resolver<InvoiceFormValues>), which is safe here since the runtime coercion behavior is correct — only the compile-time typing needed reconciling.

5.7 Miscellaneous type mismatches

Problem: A handful of smaller, self-contained type errors surfaced during a full tsc --noEmit pass: Prisma's Decimal type (for project.budget) didn't match a form component's string | null prop, and Recharts' Tooltip formatter callback could theoretically receive undefined, which the initial (value: number) => ... signature didn't allow.

Fix: Fixed with an explicit .toString() conversion at the call site for the Decimal mismatch, and by widening the Tooltip formatter's parameter type with a default of 0 for the possibly-undefined value.

5.8 An ESLint rule flagged a common "close dialog on success" pattern

Problem: Two dialog components (ClientFormDialog, ProjectFormDialog) closed themselves after a successful server action by watching the action's returned state in a useEffect and calling setOpen(false) inside it. ESLint's react-hooks/set-state-in-effect rule (part of the newer React Compiler-oriented lint rules bundled with this Next.js version) flags this as a pattern that can cause redundant renders.

Fix: Replaced the effect with React's officially recommended alternative for 'state that depends on a changing prop/value' — comparing the new state against a useState-held 'last handled state' during render and calling setOpen conditionally there, which avoids the extra render pass entirely.

5.9 Next.js 16 renamed middleware.ts to proxy.ts

Problem: The dev server ran fine but logged a deprecation warning ("The 'middleware' file convention is deprecated. Please use 'proxy' instead"), and the file also carried an export const runtime = "nodejs" left over from earlier debugging, which is no longer a valid export in the new convention ("Route segment config is not allowed in Proxy file... Proxy always runs on Node.js runtime").

Fix: Consulted the bundled Next.js docs (per this project's AGENTS.md instruction to check node_modules/next/dist/docs for breaking changes rather than relying on prior training data), renamed middleware.ts to proxy.ts, and removed the now-unnecessary/disallowed runtime export — Proxy always runs on Node.js, which conveniently also resolves the original reason that export existed (Prisma calls inside the auth-aware proxy function need a Node runtime, not Edge).

5.10 A stale Turbopack cache broke the dev server after the rename

Problem: Immediately after renaming to proxy.ts, the running dev server started throwing “Could not parse module '[project]/middleware.ts', file not found” — Turbopack's incremental cache still had a reference to the deleted file.

Fix: Stopped the dev server, deleted the .next build cache, and restarted cleanly, after which the error was gone and route protection worked correctly (verified via curl, including redirect behavior for both dashboard and portal routes).

5.11 Docker Desktop failed to start on first launch

Problem: After installing Docker Desktop, docker ps returned “Docker Desktop is unable to start.” Diagnosis showed the Docker Desktop processes were running, a hypervisor was active (so virtualization wasn't the issue), but no docker-desktop WSL2 backend distro had been registered yet — a common first-run hiccup.

Fix: This one needed the user's involvement (a GUI restart of Docker Desktop), since it wasn't something fixable from the command line. Once the user restarted Docker Desktop, docker ps responded normally and the build continued.

5.12 A minor Turbopack build warning about file tracing

Problem: next build succeeded but emitted a warning that lib/storage.ts's use of path.resolve(process.cwd(), ...) for the uploads directory could cause Turbopack's file tracer to over-include files in the production output.

Fix: Added the documented /* turbopackIgnore: true */ comment at the relevant call site, which resolved the warning on the next build with no behavior change.

6. Verification Performed

Rather than just asserting the app worked, each major flow was tested end-to-end using curl with cookie jars to simulate real browser sessions against the running dev server:

- **Login —** credentials sign-in via NextAuth's CSRF + callback flow, confirmed the session cookie grants access to /dashboard with real seeded data rendered.
- **Core pages —** clients, projects, invoices, billing, and team pages all returned real seeded records.
- **Invoice PDF —** downloaded and verified as a valid single-page PDF document.
- **File uploads —** uploaded and downloaded a test file as the owner, then confirmed an unrelated client-portal user received a 403 Forbidden on the same file — proving access control, not just the happy path.
- **Client portal scoping —** confirmed the seeded client-portal user saw only their own project, not the other client's project.
- **Role-based access control —** confirmed a MEMBER-role user is redirected away from /dashboard/team and /dashboard/settings/billing while still able to reach /dashboard/clients.
- **Static analysis —** tsc --noEmit, eslint ., and next build all pass with zero errors and zero warnings.

7. Demo Credentials

Role	Email	Password

Owner	owner@demo.com	password123
Team member	member@demo.com	password123
Client (portal only)	client@demo.com	clientpass123

8. Suggested Next Steps (Not Built, By Design)

These were intentionally left out of scope for a portfolio build, but are worth naming if this were to move toward production:

- Swap local disk file storage for S3/R2 (needed for any serverless deployment).
- Replace the mock billing flow with real Stripe Checkout + webhooks.
- Add actual email delivery for team/client invites.
- Add automated tests for permission helpers and invoice totals math.
- Move local Postgres to a hosted provider (Neon, Supabase, Railway) for deployment.